

DFCA: Decentralized Federated Clustering Algorithm

Jonas Kirch, Sebastian Becker, Tiago Koketsu Rodrigues, *Senior Member, IEEE*, Stefan Harmeling

Abstract—Clustered Federated Learning has emerged as an effective approach for handling heterogeneous data across clients by partitioning them into clusters with similar or identical data distributions. However, most existing methods, including the Iterative Federated Clustering Algorithm (IFCA), rely on a central server to coordinate model updates, typically requiring stable connectivity, synchronous communication rounds, and global aggregation of client models. These assumptions are difficult to satisfy in decentralized and heterogeneous environments, where clients may only have limited, local communication with a small subset of peers. As a result, such methods create a bottleneck and a single point of failure, limiting their applicability in realistic decentralized learning settings. This limitation is particularly severe in Internet of Things settings, where large numbers of resource-constrained devices, intermittent or sparse connectivity, and dynamic participation make reliance on a central server impractical. In this work, we introduce the Decentralized Federated Clustering Algorithm (DFCA), a fully decentralized clustered federated learning algorithm that enables clients to collaboratively train cluster-specific models without central coordination. DFCA uses a sequential running average to aggregate models from neighbors as updates arrive, providing a communication-efficient alternative to batch aggregation while maintaining clustering performance. Our experiments on various datasets demonstrate that DFCA outperforms other decentralized algorithms and performs comparably to centralized IFCA, even under sparse connectivity, highlighting its robustness and practicality for dynamic real-world decentralized networks.

Index Terms—Machine Learning, Federated Learning, Clustered Learning, Decentralized Optimization

I. INTRODUCTION

FEDERATED Learning (FL) has emerged as a new paradigm that allows for clients to train Machine Learning (ML) models collaboratively without the need to share their raw data. By enabling collaborative training across multiple devices, FL has gained significant attention in research and industry, especially since distributed computing with different devices has become a crucial component of modern technology. The most known FL implementation strategy, FedAvg [32], and most other known FL algorithms assume a setting with a central instance that aggregates the local updates of all clients to form a global model, which is then broadcast

back to the network. While effective, this orchestration with a central server introduces several limitations, including a single point of failure, communication delays, and bottlenecks that are often connected to more challenging learning settings with Internet of Things (IoT) devices and mobile phones [22]. In such environments, devices are often highly heterogeneous in data or computation, network connectivity is intermittent, and communication links are unreliable. These characteristics amplify the central server bottleneck, making synchronous aggregation inefficient or even infeasible in practical IoT deployments, thereby motivating decentralized approaches.

To address the limitations of centralized Federated Learning (CFL), recent research has explored decentralized Federated Learning (DFL), where clients communicate with each other without the need for a central instance [22]. Decentralized strategies often utilize peer-to-peer (P2P) [21] or gossip-based [14, 13] exchange methods to achieve convergence through direct communication between clients. DFL approaches remove the single point of failure, often reduce communication cost and delays, and improve the overall robustness [49].

Concurrently, clustered FL has appeared as a proposed solution to data heterogeneity across clients, another major issue in ML and FL. In most real-world scenarios, the data is not independently identically distributed (non-IID) over all clients, making global aggregation less efficient and suboptimal. Clustered FL methods attempt to cluster clients into groups with similar data distributions, allowing clusters to capture local patterns and characteristics during training [39]. The most popular among clustered FL techniques is the Iterative Federated Clustering Algorithm (IFCA) [11], which is a centralized, training loss-based clustering method, where clusters of clients are evaluated locally after each global training round. As most other clustered FL techniques that have been developed over the recent years also presume central instance coordination, they are not optimized for decentralized learning settings. In this paper, we propose the Decentralized Federated Clustering Algorithm (DFCA) to address this issue.

Our contributions:

- 1) We formulate DFCA, a fully decentralized federated clustering algorithm inspired by IFCA and designed to operate effectively in low-connectivity networks with heterogeneous client data distributions, often seen in IoT deployments.
- 2) We incorporate a sequential running-average parameter exchange strategy that preserves clustering performance while enabling communication-efficient updates across the network.

Jonas Kirch is with the Graduate School of Information Science at Tohoku University, Sendai, Japan. Formerly, he was with the Fraunhofer Institute for Software and Systems Engineering, Dortmund, Germany.

Sebastian Becker is with the Fraunhofer Institute for Software and Systems Engineering, Dortmund, Germany.

Tiago Koketsu Rodrigues is with the Graduate School of Information Science at Tohoku University, Sendai, Japan.

Stefan Harmeling is with the Department of Computer Science at TU Dortmund University and with the Lamarr Institute for Machine Learning and AI, Dortmund, Germany.

- 3) Through extensive experiments on various datasets, we demonstrate that DFCA matches the accuracy of the centralized IFCA baseline and outperforms decentralized alternatives. Furthermore, sequential aggregation achieves performance comparable to synchronous batch aggregation, highlighting its practicality for real-world decentralized settings.

After looking at the related work and problem formulation in Sections II and III, we proceed to introduce our method in Section IV, analyze its convergence in Section V, and show our simulation results in Section VI. We will finally conclude this paper's findings in Section VII.

II. RELATED WORK

A. Decentralized Federated Learning

DFL originated from decentralized Stochastic Gradient Descent (SGD) optimization [26] and was later formulated by Lalitha *et al.* [22] as a distinct concept for FL. During the following years, researchers proposed new frameworks and concepts around DFL, leading to rapid growth of the field. Research aspects of DFL include network topologies [47, 34, 29, 31, 6], communication protocols [44, 21, 13, 18, 14, 2] and iteration orders [49]. Explicit DFL paradigms [5, 41, 40, 15, 48, 1, 37, 36, 42, 7, 46] then put these concepts and assumptions in the context of real-world learning settings. However, there still remains a gap in performance between CFL and DFL [45], especially in low-connectivity settings and in the presence of heterogeneity, which motivated us to have a closer look into decentralized optimization.

B. Clustered FL

First being introduced by Sattler *et al.* [39] in 2019, clustered FL addresses the issue of handling heterogeneous data distributions of clients in a network. To optimize performance and adapt to different learning settings, researchers have introduced different methods to cluster the clients into groups with similar data distributions [10]. After the initial introduction of client-side clustered FL algorithms based on client loss minimization [39, 11], recent publications have focused on optimizing this strategy in different learning contexts [30, 24, 16]. Voting-scheme-based [12] or k-means-based [28] methods are alternative solutions utilizing client-side clustering. In contrast to the approaches mentioned above, our algorithm works in decentralized, low-connectivity settings without the need for a central instance. Instead of implicitly evaluating neighboring clients, as in Onoszko *et al.* [35], DFCA explicitly groups clients according to their data distributions. Lin *et al.* [27] highlighted the potential of decentralized federated clustering methods when they introduced their decentralized soft-clustering algorithm for scenarios in which clients possess multiple data distributions. However, their method, called FedSPD, addresses a soft clustering scenario where each client may hold data from multiple distributions simultaneously, ignoring hard clustering scenarios where each client only has access to one data distribution. This motivated us to develop a decentralized approach capable of matching the performance of centralized IFCA in low-connectivity settings

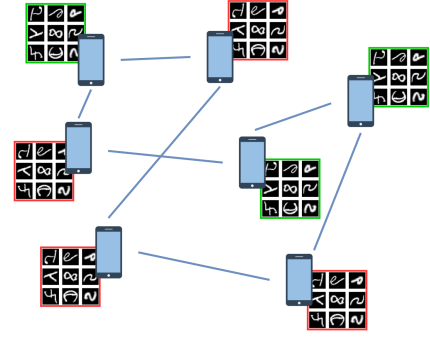


Fig. 1: Illustration of the DFCA problem for Rotated EMNIST with two different data distributions

with clients holding different data distributions, as described later in Section III.

III. PRELIMINARIES

Let M be a set of N clients that are connected to each other in a graph. We represent the graph by N sets $\mathcal{N}_i \subset M$, which contain the neighboring clients for each client i (i.e., neighborhood sets). The clients are partitioned into k disjoint clusters $\mathcal{S}_1, \dots, \mathcal{S}_k \subset M$. Each cluster is associated with a distinct data distribution $\mathcal{D}_1, \dots, \mathcal{D}_k$. Our problem setup is illustrated in Figure 1, which shows the different data distributions represented by handwritten character digits (EMNIST) rotated by 0, 90, 180, 270 degrees.

For each client i , we sample a data set D_i distributed according to $\mathcal{D}_j$ of the associated cluster j meaning that each client has data from one of k data distribution. Additionally, at each client i we store all k Machine Learning models (ML-models), which are parameterized by $\theta_{i,j}$ where $j \in [k] := \{1, \dots, k\}$ and k is the number of clusters. Client i will update the parameters $\theta_{i,j}$ of the model, which is associated with its corresponding cluster j , by gradient descent using D_i . During aggregation (communication phase) the local models for all clusters are updated using the models of the neighboring clients. Note that the corresponding (assigned) cluster of a client might change after an iteration.

For client-local learning, we consider a loss function $\mathcal{L}(\theta_{i,j}, d)$ that calculates the loss for a single data point $d \in D_i$. These losses can be combined on the client-level and also on the cluster-level:

Let's first consider the loss for an individual client. We assume that client i is assigned to cluster j . Then we write the client-specific objective as

$$F_{\text{client}}(\theta_{i,j}, D_i) = \frac{1}{|D_i|} \sum_{d \in D_i} \mathcal{L}(\theta_{i,j}, d). \quad (1)$$

Second, we define the loss for each cluster j as the sum of the losses of the associated clients,

$$F_{\text{cluster}}(j) = \sum_{i \in \mathcal{S}_j} F_{\text{client}}(\theta_{i,j}, D_i). \quad (2)$$

Finally, we define the **global loss**, which combines all data points across all clients into a single number:

$$F_{\text{global}} = \sum_{j=1}^k F_{\text{cluster}}(j) \quad (3)$$

Having formulated the loss functions on client- and cluster-level, we next introduce the decentralized learning algorithm DFCA, which allows clients to collaboratively minimize their respective cluster-specific losses while communicating with their neighbors in the graph to exchange results.

IV. DECENTRALIZED FEDERATED CLUSTERING ALGORITHM

Algorithm 1 Decentralized Federated Clustering Algorithm (DFCA)

```

1: Input: number of clusters  $k$ , number of iterations  $T$ 
2: Local: step size  $\gamma$ , number of local epochs  $\tau$ 
3:
4: DFCA-GI: initialize  $\theta_{i,j}$  per cluster and publish models to all clients
5: DFCA-LI: initialize  $\theta_{i,j}$  for all clusters per client (personalized models)
6:
7: for  $t = 0, 1, \dots, T - 1$  do
8:    $M_t \leftarrow$  subset of worker machines (participating devices)
9:   for worker machine  $i \in M_t$  do
10:
11:     Step 1: AssignCluster
12:      $c(i) \leftarrow \arg \min_j F_{\text{client}}(\theta_{i,j}, D_i)$ 
13:        $\triangleright$  run local inference on all models
14:
15:     Step 2: LocalUpdate
16:     for  $q = 0, \dots, \tau - 1$  do
17:        $\theta_{i,c(i)} \leftarrow \theta_{i,c(i)} - \gamma \nabla F_{\text{client}}(\theta_{i,c(i)}, D_i)$ 
18:          $\triangleright$  stochastic gradient descent
19:     end for
20:
21:     Step 3: Aggregation
22:     for each cluster  $j = 1, \dots, k$  do
23:        $r \leftarrow 0$ 
24:       for each neighbor  $m \in \mathcal{N}_{i,j}$  do
25:          $r \leftarrow r + 1$ 
26:          $\theta_{i,j} \leftarrow \frac{r}{r+1} \theta_{i,j} + \frac{1}{r+1} \theta_{m,j}$ 
27:          $\triangleright$  running average for each cluster
28:       end for
29:     end for
30:   end for
31: end for

```

DFCA starts with initialization of the model parameters and then iterates three steps: (1) Cluster Assignment, (2) Local Updates, and (3) Decentralized Aggregation. Steps (1) and (2) are similar to existing training loss based, client-side clustered FL algorithms ([11, 27, 10]). Step (3) enables decentralized learning.

Initialization. Before detailing the three iterative steps, we explain how the model parameters $\theta_{i,j}$ are initialized. We consider two variants: (i) with the **global initialization method (DFCA-GI)**, all k models are centrally generated and published via broadcast (or initialized locally using the same seed) before the first iteration, so that **every client holds the same model parameters at the beginning**. (ii) For the **local initialization method (DFCA-LI)**, all clients start on different parameters, i.e., **each client can initialize the models locally**.

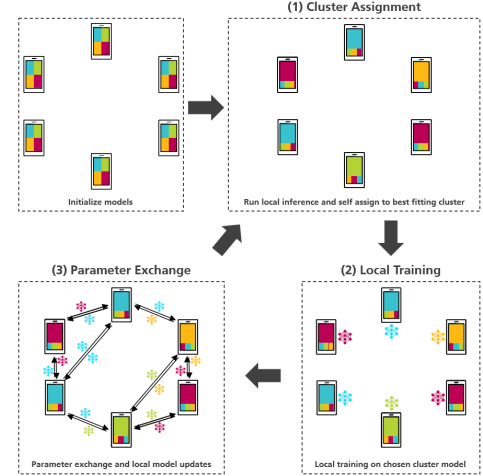


Fig. 2: After initialization, DFCA iterates three steps: (1) cluster assignment, (2) local training, and (3) parameter exchange

A. Cluster Assignment

Every client i is assigned to cluster $c(i) \in [k]$ through inference on the current parameters $\theta_{i,j}$. More formally, we update $c(i)$ to be the **argmin of the local client loss**,

$$c(i) \leftarrow \arg \min_j F_{\text{client}}(\theta_{i,j}, D_i). \quad (4)$$

Hereby, the overall loss F_{global} is non-increasing. **These cluster assignments are repeated at the start of each training loop.**

B. Local Update

The local update at client i runs several epochs at the client-level using (stochastic) gradient descent on the local data D_i with respect to $\theta_{i,c(i)}$:

$$\theta_{i,c(i)} \leftarrow \theta_{i,c(i)} - \gamma \nabla F_{\text{client}}(\theta_{i,c(i)}, D_i) \quad (5)$$

(with learning rate γ), i.e., we **only modify the parameters of the assigned cluster $c(i)$** . Again, the gradient descent ensures that the global loss F_{global} is decreasing (at least in expectation).

C. Decentralized aggregation (a.k.a. communication step)

The goal of our algorithm is that at the end, **all clients hold all k trained models**. Thus, limiting the communication to neighbors within the *same* cluster would be suboptimal. Instead, **all clients exchange their parameters with all of their neighbors according to the graph** and locally average the

models. More formally, client $i \in M$ receives parameters from all neighbors in its neighborhood $\mathcal{N}_i$. To maintain cluster-specific updates in a sparse decentralized network, clients receive models from their neighbors but only send out the model parameters $\theta_{i,c(i)}$ that they trained themselves in the previous step.

To specify the aggregation equations, we split the neighbors of client i according to their cluster assignments:

$$\mathcal{N}_{i,j} := \{m \mid m \in \mathcal{N}_i \text{ and } c(m) = j\} \subset \mathcal{N}_i \quad (6)$$

(for $i \in [N]$ and $j \in [k]$). In this phase, the clients update the parameter sets for all clusters, not only the one of their assigned cluster $c(i)$.

a) *Batch aggregation*: Next, we define the batch update (synchronous), which assumes that all neighbors m have reported their current models $\theta_{m,j}$:

$$\theta_{i,j} \leftarrow \frac{1}{|\mathcal{N}_{i,j}| + 1} \left(\theta_{i,j} + \sum_{m \in \mathcal{N}_{i,j}} \theta_{m,j} \right) \quad (7)$$

(for $i \in [N]$ and $j \in [k]$).

b) *Sequential aggregation*: While batch aggregation is the perfect scenario, in practice, neighbors might report their updates asynchronously, and we can never be sure whether a client has disconnected or not. Thus we need sequential averaging that is robust against failing clients and random arrival times. The basic idea is to replace the averaging in Eq. 7 with an online version (a.k.a. *running average*): we start with the local parameter value $\theta_{i,j}$ and update it as the messages from the other clients come in. Assuming r neighbors have already reported their updates for cluster j , we update $\theta_{i,j}$ with:

$$\theta_{i,j} \leftarrow \frac{r}{r+1} \theta_{i,j} + \frac{1}{r+1} \theta_{m,j} \quad \text{for } r \in [|\mathcal{N}_{i,j}|] \quad (8)$$

(for $i \in [N]$ and $j \in [k]$).

Our sequential aggregation naturally supports asynchronous updates, allowing each client to integrate neighbor models immediately as they arrive, which can improve efficiency and reduce idle time in fully distributed deployments. This approach is also memory efficient, as it only requires storing the current estimate per model rather than all neighbor updates. Moreover, using a running average ensures that each incoming model contributes proportionally to the aggregated model, providing a stable and principled approximation of the full batch aggregation even in dynamic and sparse networks.

D. Computational and communication overhead.

At each iteration, a client performs local training only on its currently assigned cluster-specific model, resulting in a computational cost comparable to standard decentralized SGD. Although each client stores all k models, inference-based cluster assignment requires only forward passes and results in overhead compared to local training. Communication is fully decentralized and limited to parameter exchanges with neighboring clients without relying on global broadcasts and, although not specifically tested in our experiments, global synchronization. The per-round communication cost therefore

TABLE I: Notation and parameters used in DFCA convergence analysis

Symbol	Description
N	Number of clients in the network
$M = \{1, \dots, N\}$	Set of clients
k	Number of clusters
$[k]$	Cluster index set $\{1, \dots, k\}$
$\mathcal{S}_j$	Set of clients belonging to cluster j
$\mathcal{D}_j$	Data distribution of cluster j
D_i	Local dataset of client i
d	Data sample
$\mathcal{L}(\theta, d)$	Sample-wise loss function
$F_{\text{client}}(\theta, D_i)$	Empirical loss of client i
$F_{\text{cluster}}(j)$	Objective of cluster j
F_{global}	Global objective $\sum_{j=1}^k F_{\text{cluster}}(j)$
$\theta_{i,j}^t$	Model of client i for cluster j at round t
Θ_j^t	Stacked models $(\theta_{1,j}^t, \dots, \theta_{N,j}^t)$
$\bar{\theta}_j^t$	Network-wide average model for cluster j
$c_t(i)$	Cluster assignment of client i at round t
$c_\star(i)$	Ground-truth cluster assignment of client i
$G = (M, E)$	Communication graph
$\mathcal{N}_i$	Neighborhood of client i in G
W	Mixing matrix (synchronous gossip)
$W_t^{(j)}$	Time-varying mixing matrix for cluster j
λ	Consensus contraction factor (synchronous case)
$\tilde{\lambda}$	Windowed contraction factor (asynchronous case)
B	Gossip window size in asynchronous setting
Disp_j^t	Disagreement for cluster j at round t
E_t	Disagreement matrix $X_t - \mathbf{1}\bar{x}_t^\top$
γ	Learning rate (step size)
$g_{i,j}(\theta)$	Stochastic gradient at client i , cluster j
L	Smoothness constant
σ^2	Variance bound of stochastic gradients
μ	PL constant (when PL condition holds)
δ	Cluster separability margin (Assumption A5)
τ	Stabilization time of assignments

scales linearly with the neighborhood size and the number of clusters k . In typical IoT deployments, where both k and neighborhood sizes are relatively small, this results in manageable overhead, making DFCA suitable for scenarios in which low communication cost is a key requirement [9].

V. CONVERGENCE SUMMARY

We briefly summarize the convergence properties of DFCA. Full proofs are deferred to Appendix A.

a) *Setup*: Each client stores all k models $\{\theta_{i,j}^t\}$ here with index t for the round. Each round executes three steps: (i) cluster assignment by local inference, (ii) local stochastic gradient descent on the assigned model, and (iii) decentralized aggregation with neighbors. Aggregation is carried out via gossip (either synchronous averaging or sequential running averages), which preserves the network-wide average and contracts disagreement among clients. For cluster j , define the stacked vector $\Theta_j^t = (\theta_{1,j}^t, \dots, \theta_{N,j}^t)$ and the network average $\bar{\theta}_j^t = \frac{1}{N} \sum_{i=1}^N \theta_{i,j}^t$. We measure per-cluster *disagreement* by $\text{Disp}_j^t = \frac{1}{N} \sum_{i=1}^N \|\theta_{i,j}^t - \bar{\theta}_j^t\|^2$. The three steps of one DFCA update round can be written as:

- 1) **Assignment**: $c_t(i) = \arg \min_{j \in [k]} F_{\text{client}}(\theta_{i,j}^t, D_i)$.
- 2) **Local descent (assigned index only)**:

$$\theta_{i,c_t(i)}^{t+\frac{1}{2}} = \theta_{i,c_t(i)}^t - \gamma g_{i,c_t(i)}(\theta_{i,c_t(i)}^t), \quad (9)$$

with $\theta_{i,j}^{t+\frac{1}{2}} = \theta_{i,j}^t$ ($j \neq c_t(i)$) and stochastic gradient $g_{i,j}$.

3) **Decentralized aggregation (all j):**

$$\theta_{i,j}^{t+1} = \sum_{m \in \{i\} \cup \mathcal{N}_i} w_{im,t}^{(j)} \theta_{m,j}^{t+\frac{1}{2}}, \quad (10)$$

where $W_t^{(j)} = (w_{im,t}^{(j)})$ respects G , is row-stochastic, and is doubly-stochastic in the synchronous (batch) case. In the sequential/asynchronous case, $W_t^{(j)}$ is time-varying with standard joint-connectivity.

We adopt the following standard assumptions.

- (A1) **Smoothness.** For all i, j , $F_{\text{client}}(\cdot, D_i)$ is L -smooth.
- (A2) **Noise.** Unbiased stochastic gradients with bounded variance: $\mathbb{E}[g_{i,j}(\theta) \mid \theta] = \nabla F_{\text{client}}(\theta, D_i)$, $\mathbb{E}\|g_{i,j}(\theta) - \nabla F_{\text{client}}(\theta, D_i)\|^2 \leq \sigma^2$.
- (A3) **Graph mixing.** In the synchronous case there exists a symmetric, doubly-stochastic W respecting G with spectral gap $1 - \lambda > 0$ such that $\|XW - \mathbf{1}\bar{x}^\top\| \leq \lambda \|X - \mathbf{1}\bar{x}^\top\|$ for any row-stacked X . In the asynchronous case, $\{W_t^{(j)}\}$ are row-stochastic, edges are repeatedly activated with bounded delays, and there exists a window B and $\tilde{\lambda} \in (0, 1)$ such that over any B consecutive rounds disagreement contracts by $\tilde{\lambda}$. (extensive formulation can be found in Appendix A0b)
- (A4) **Objective curvature.** Either (PL) each $F_{\text{cluster}}(j; \cdot)$ satisfies the μ -Polyak–Łojasiewicz (PL) inequality, or (Cvx) each is convex.
- (A5) **Separability (IFCA-style).** There exists $\delta > 0$ such that, in a neighborhood of the cluster minimizers $\{\theta_j^*\}_{j=1}^k$, the argmin-of-loss assignment selects the true cluster:

$$\mathbb{E}_{d \sim \mathcal{D}_{c(i)}}[\mathcal{L}(\theta_{i,c(i)}, d)] \leq \min_{j \neq c(i)} \mathbb{E}_{d \sim \mathcal{D}_{c(i)}}[\mathcal{L}(\theta_{i,j}, d)] - \delta. \quad (11)$$

On a high-level, the analysis combines two ingredients:

- 1) *Cluster assignment:* Choosing the best-fitting model index per client never increases the global loss, and after sufficient descent the assignments stabilize to the ground-truth clusters.
- 2) *Local descent + gossip:* Gradient descent decreases the cluster objectives, up to stochastic noise and a *disagreement penalty*. Gossip averaging preserves the average model and contracts disagreement at a rate governed by the graph spectral gap.

Together, these steps imply that DFCA behaves like k independent instances of decentralized SGD, one per cluster, after a finite burn-in.

Theorem 1 (Convergence of DFCA). *Assume (A1)–(A5), choose $\gamma \leq c/L$ for a small numerical constant c , and let λ (resp. $\tilde{\lambda}$) be the consensus factor in the synchronous (respectively asynchronous) case. Then:*

- (i) (Pre-stabilization) F_{global}^t is non-increasing in expectation across assignment and local steps. The disagreements $\{\text{Dis}_j^t\}$ remain bounded and contract at rate λ (or $\tilde{\lambda}$ over windows).
- (ii) (Stabilization) There exists $\tau < \infty$ such that $c_t(i) = c_\star(i)$ for all $t \geq \tau$.

(iii) (Post-stabilization) For $t \geq \tau$, DFCA is k independent copies of decentralized SGD on $F_{\text{cluster}}(j)$.

- Under (PL) for some $\mu > 0$,

$$\mathbb{E}[F_{\text{global}}^{\tau+T} - F_{\text{global}}^\star] \leq (1 - \mu\gamma/2)^T C_0 + O\left(\frac{\gamma\sigma^2}{\mu}\right) + O\left(\frac{\gamma L}{1-\lambda}\sigma^2\right), \quad (12)$$

with C_0 depending on the gap at $t = \tau$; in *asyn*, replace $(1 - \lambda)$ by the windowed $(1 - \tilde{\lambda})$.

- Under (Cvx),

$$\frac{1}{T} \sum_{t=\tau}^{\tau+T-1} \sum_{j=1}^k \mathbb{E}\|\nabla F_{\text{cluster}}(j; \bar{\theta}_j^t)\|^2 \leq O\left(\frac{F_{\text{global}}^\tau - F_{\text{global}}^\star}{\gamma T}\right) + O(\gamma L \sigma^2) + O\left(\frac{\gamma L}{1-\tilde{\lambda}}\sigma^2\right), \quad (13)$$

and choosing $\gamma = \Theta(1/\sqrt{T})$ yields the usual $O(1/\sqrt{T})$ rates (with the consensus penalty).

b) *Takeaway:* DFCA converges at essentially the same rate as decentralized SGD, up to an additional term reflecting network connectivity. Crucially, **all clients obtain all k cluster models despite decentralized, asynchronous communication**. The appendix provides a detailed proof by combining IFCA's cluster-assignment arguments with standard decentralized SGD analyses.

VI. EXPERIMENTS

Next, we present our experiments with DFCA in practical learning settings. As common in the clustered FL literature [11, 27, 38], we conduct experiments on the MNIST [20], EMNIST [8], CIFAR-10 [23], and FEMNIST [4] datasets, while applying rotations to the data to create different distributions. Our method is compared to the decentralized soft-clustering method FedSPD [27] and the optimized Decentralized Federated Averaging algorithm DFedAvgM [44]. IFCA [11] serves as the centralized baseline. After providing results for additional experiments with different connection probabilities, we discuss the communication efficiency and analyze the results of our experiments.

A. Experimental Setting

EMNIST: For the training with EMNIST [8] (balanced split), we use $N = 100$ clients for $k = 2$ and $N = 200$ for $k = 4$ clusters and simulate two or four different data distributions by augmenting the datasets, applying 0, 180 or 0, 90, 180, 270 degree rotations to the data. The Convolutional Neural Network (CNN) used for training contains two convolutional layers, each followed by a ReLU activation function, a max-pool layer, and a batch normalization layer. The models are trained for $\tau = 5$ local epochs with a learning rate of $\gamma = 0.1$, using Stochastic Gradient Descent over $T = 150$ global iterations. For the connection between clients, we use the adjacency matrix of an Erdős–Rényi graph with a connection probability of 0.15. All experiments are run on five random seeds, with the metric values being averaged over all runs.

MNIST: The training with MNIST [20] is conducted on $N = 240$ clients and $k = 4$ clusters and data distributions (0, 90, 180, 270 degree rotations). We use a simple Multilayer Perceptron (MLP) with one hidden layer of size 2048 followed by a ReLU activation function. The other training parameters stay consistent with the EMNIST experimental setting, with the exception of reducing the connection probability to 0.1.

CIFAR-10: The setup for our experiments with the CIFAR-10 [23] dataset is similar to the EMNIST setup. We train with an identical CNN architecture over $N = 50$ clients and $k = 2$ clusters. We change the learning rate to $\gamma = 0.25$ and the graph connection probability to 0.2.

FEMNIST: To test the algorithm in settings with even higher heterogeneity, we conducted experiments on the FEMNIST [4] dataset. The training is done on $N = 400$ clients, who each get data from one distinct writer, with $k = 3, 4, 10$ clusters with a graph connection probability of 0.2 and all other parameters equal to the MNIST experiments.

B. General Accuracy Analysis

Our experiments demonstrate that DFCA consistently outperform the decentralized baselines FedSPD and DFedAvgM while achieving accuracy comparable to the centralized IFCA algorithm (Table II). The reason for FedSPD underperforming is that it represents a decentralized FL approach that adapts a single local model per client via regularization-based decentralized averaging. In the presence of clustered heterogeneity, this limits effective model sharing among clients with similar data distributions, whereas DFCA explicitly maintains shared cluster-specific models. The plots in Figure 4 additionally show DFCA-GI converging at a similar rate as IFCA, while DFCA-LI converges slower but steeper than the other two methods. In more heterogeneous settings with larger numbers of clients (MNIST, EMNIST, FEMNIST; Table III), DFCA maintains competitive performance, indicating that the sequential aggregation strategy effectively preserves cluster-specific models even as heterogeneity increases. IFCA's unusually high standard deviation for the EMNIST experiments with $k = 4$ occurs because IFCA detected only three clusters in one of its five runs. The topic of misclusterings is further discussed in the next sections.

Insights. In DFL, the way clients exchange model updates plays a crucial role in both convergence and efficiency. Beyond simple averaging, enabling clustered FL in decentralized networks is particularly valuable, as it allows clients with heterogeneous data to specialize in distinct model clusters without relying on a central coordinator. The general advantages of DFL, such as improved scalability, resilience to single points of failure, and better suitability for bandwidth-limited or peer-to-peer networks, have already been highlighted in prior works [22, 21, 49]. The results in Tables II and III show that DFCA not only outperforms the decentralized baselines but also does not fall short when compared to centralized IFCA. Despite evidence that DFL lags behind CFL [45], we reduce the accuracy difference to about 1% in CFL's favor, including in non-IID and low-connectivity settings.

C. Noisy Feature Skew Analysis

We further assess the robustness of DFCA under more challenging heterogeneous feature distributions, by introducing two additional scenarios and varying the dominance ratio $\alpha \in \{0.7, 0.8, 0.9\}$ (with $\alpha = 1$ matching the main setup). Such feature skews can be seen in IoT and distributed sensing applications, where devices operate under heterogeneous and dynamically changing conditions (e.g., sensor noise, calibration differences, environmental effects, or device-specific preprocessing), leading to partially misaligned feature transformations across clients. The setup is distributed in two different styles:

1. **Cluster-consistent skew:** Clients belonging to the same cluster share the same dominant transformation, while retaining a non-dominant share ($1 - \alpha$) of samples with a different transformation. This setting models coherent subpopulations with strong intra-cluster similarity.

2. **Cluster-inconsistent skew:** Each client receives its non-dominant transformation independently at random, introducing a more challenging level of noise to the clusters. This represents heterogeneous environments in which local feature distributions are not aligned across clients.

The results of our experiments are illustrated in Table IV and Figure 5.

Insights. DFCA-LI is more robust than both DFCA-GI and IFCA under more noisy feature distribution settings, where clusters are less clear. The method clusters more accurately under most settings, and outperforms all other decentralized methods and in most cases even the centralized baseline IFCA in terms of accuracy. The accuracy gains are largely due to the instability observed in IFCA and DFCA-GI, whose clustering assignments degrade substantially under higher noise levels (see Figure 5 and Table IV).

D. Connectivity Analysis

Figure 7 shows the test accuracy of DFCA-LI and DFCA-GI under different, fixed connectivity settings on EMNIST. There, we can observe that a connectivity of 0.15 is sufficient and the test accuracy does not change significantly when further increasing the connectivity rate. In settings with connectivity probabilities below 0.1, DFCA-LI attains slightly lower accuracies than DFCA-GI, which can be attributed to its slower convergence caused by the additional time required for clustering, as seen in Figures 4 and 5.

Insights. DFCA leverages a sequential running average to integrate neighbor updates efficiently, avoiding the need to store all incoming models and allowing updates to proceed asynchronously as they arrive. As a result, it improves scalability and robustness to network sparsity, while still achieving accuracy comparable to the centralized IFCA baseline.

E. Misclusterings and finding the right k

When the number of data distributions is known a priori, as often considered in the literature [11], misclusterings are rather rare. During the experiments, we saw that DFCA is more robust to misclusterings than IFCA, as IFCA sometimes

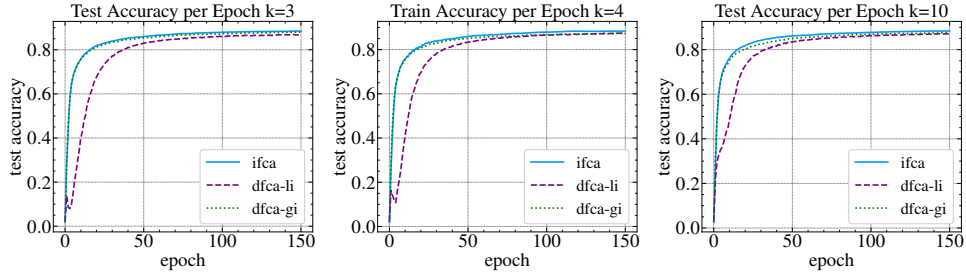


Fig. 3: Graphs for FEMNIST experiments with different values for k

TABLE II: Results of experiments with EMNIST ($N = 100$ clients) and CIFAR-10 ($N = 50$ clients)

Dataset	DFL				CFL
	DFCA-GI (ours)	DFCA-LI (ours)	FedSPD	DFedAvgM	IFCA
MNIST	93.7 ± 0.07	92.9 ± 0.06	86.2 ± 1.52	91.4 ± 0.21	93.9 ± 0.06
EMNIST	85.7 ± 0.13	85.3 ± 0.09	79.7 ± 0.92	73.5 ± 1.19	85.7 ± 0.11
CIFAR-10	81.5 ± 0.40	80.4 ± 0.22	78.9 ± 0.23	76.0 ± 0.96	82.5 ± 0.11

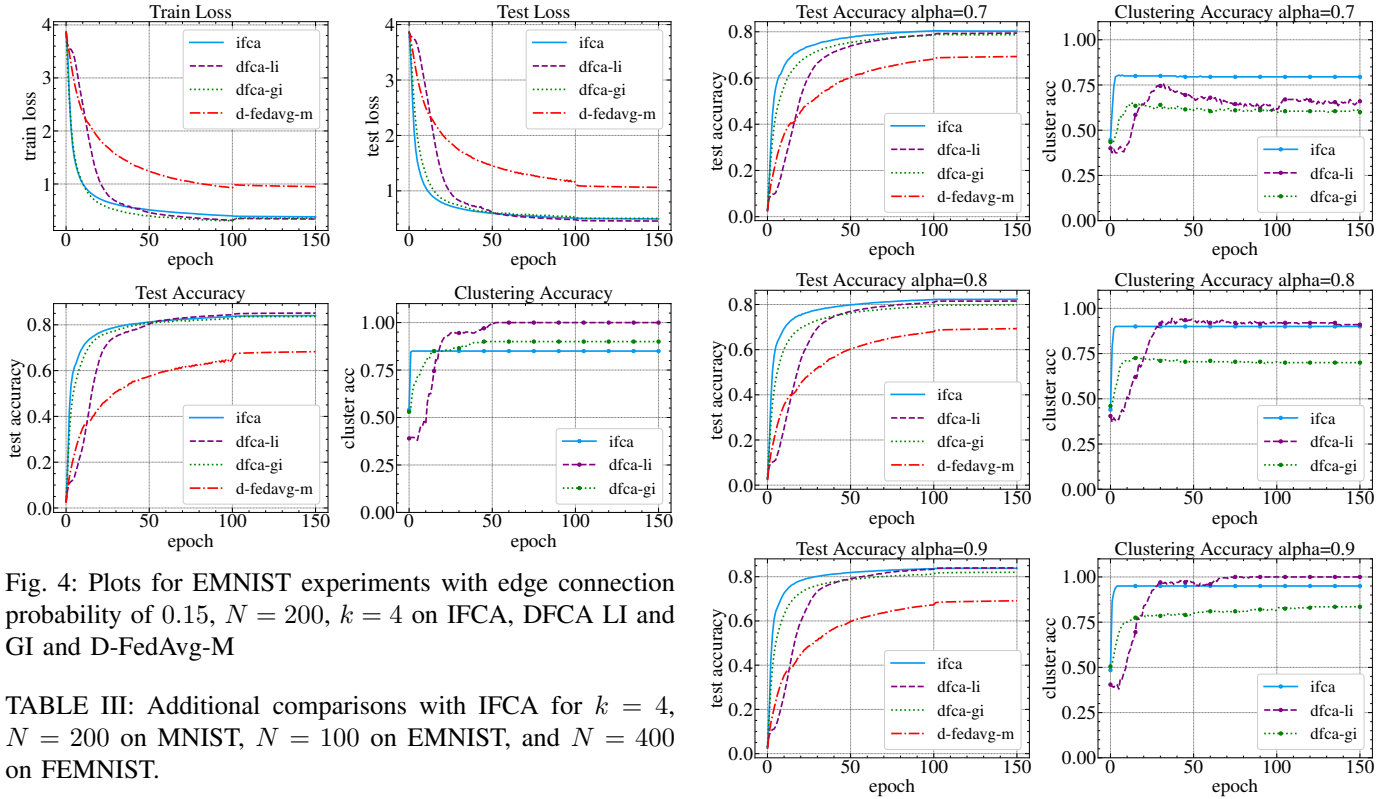


Fig. 4: Plots for EMNIST experiments with edge connection probability of 0.15, $N = 200$, $k = 4$ on IFCA, DFCA LI and GI and D-FedAvg-M

TABLE III: Additional comparisons with IFCA for $k = 4$, $N = 200$ on MNIST, $N = 100$ on EMNIST, and $N = 400$ on FEMNIST.

Dataset	DFL		CFL
	DFCA-GI (ours)	DFCA-LI (ours)	IFCA
MNIST	92.8 ± 0.63	92.4 ± 0.22	93.1 ± 0.73
EMNIST	83.6 ± 2.10	85.1 ± 0.10	84.0 ± 1.82
FEMNIST	87.1 ± 0.30	86.4 ± 0.15	88.2 ± 0.11

struggled with finding all correct clusters for 4 data distributions. IFCA only found three of four clusters in one to two out of five runs, which resulted in lower accuracy than DFCA in the EMNIST results. However, even when IFCA misclusters, the performance remains competitively high with only marginal differences in accuracy.

Insights. In the case of natural feature skew, like in FEMNIST, we started with $k = 10$ for $N = 400$ clients, where each client receives data from one distinct writer. In

Fig. 5: Results for cluster-inconsistent noisy feature skew experiments on EMNIST with connection probability of 15%, $k = 4$, $\alpha = 0.8$ and $\alpha = 0.9$

the experiments, we saw that all methods (DFCA-LI, DFCA-GI, IFCA) detected between three and five clusters. This occurs because the feature differences between writers are often subtle, making it difficult for the models to reliably distinguish fine-grained variations. As a result, they tend to generalize across writers and merge them into fewer effective clusters. The comparisons of performance for $k = 3, 4, 10$ can be seen in Table V and in further plots in the appendix, where the differences in performance are not substantial. Using $k = 3$

TABLE IV: Experiments with EMNIST ($N = 200$ clients) and on noisy feature distributions

Setting	DFL			CFL
	DFCA-GI (ours)	DFCA-LI (ours)	DFedAvgM	IFCA
Consistent, $\alpha = 0.7$	80.5 $\pm$ 1.11	82.9 $\pm$ 0.11	68.2 $\pm$ 0.30	82.4 $\pm$ 1.01
Consistent, $\alpha = 0.8$	80.5 $\pm$ 1.11	82.9 $\pm$ 0.11	68.2 $\pm$ 0.30	82.4 $\pm$ 1.08
Consistent, $\alpha = 0.9$	82.8 $\pm$ 1.05	83.9 $\pm$ 0.27	68.3 $\pm$ 0.35	82.9 $\pm$ 1.78
Inconsistent, $\alpha = 0.7$	79.8 $\pm$ 1.47	81.5 $\pm$ 1.64	69.3 $\pm$ 0.24	82.3 $\pm$ 1.58
Inconsistent, $\alpha = 0.8$	79.8 $\pm$ 1.46	81.5 $\pm$ 1.64	69.3 $\pm$ 0.24	82.3 $\pm$ 1.57
Inconsistent, $\alpha = 0.9$	82.0 $\pm$ 1.64	84.0 $\pm$ 0.15	69.1 $\pm$ 0.31	83.8 $\pm$ 1.73

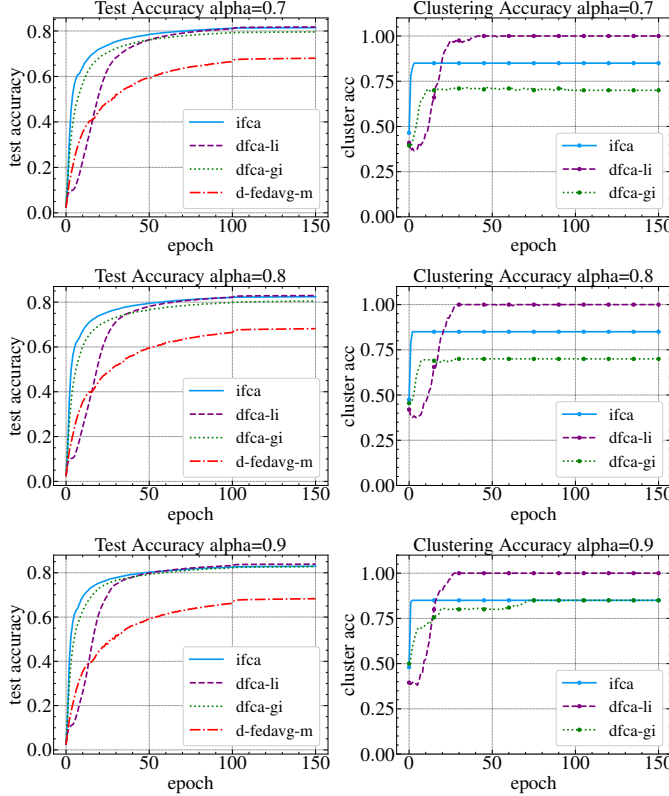


Fig. 6: Results for cluster-consistent noisy feature skew experiments on EMNIST with connection probability of 15%, $k = 4$, $\alpha = 0.8$ and $\alpha = 0.9$

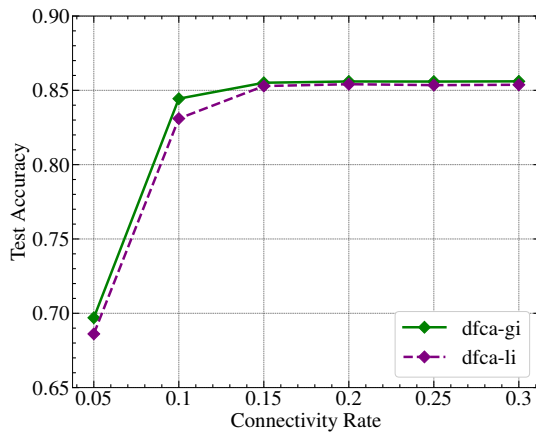


Fig. 7: Test Accuracy of DFCA-LI and DFCA-GI under different connectivity settings (EMNIST, $k = 2$, $N = 100$)

instead of $k = 10$ substantially reduces the communication and memory cost, as less models need to be saved and maintained.

TABLE V: Results on FEMNIST for different k with $N = 400$ clients, each holding data from a distinct writer

Num Clusters	DFL		CFL
	DFCA-GI (ours)	DFCA-LI (ours)	IFCA
$k = 3$	87.9 $\pm$ 0.59	86.9 $\pm$ 0.89	88.5 $\pm$ 0.52
$k = 4$	87.3 $\pm$ 0.62	87.3 $\pm$ 0.27	88.4 $\pm$ 0.12
$k = 10$	87.5 $\pm$ 0.76	87.1 $\pm$ 0.79	88.3 $\pm$ 0.43

VII. CONCLUSION

a) Conclusion: In this work, we introduced DFCA, a fully serverless method inspired by IFCA, that allows cluster-specific models to emerge and propagate through heterogeneous, sparse peer-to-peer networks. By employing a sequential running-average aggregation scheme, DFCA leverages stable learning with high clustering accuracy in heterogeneous environments where centralized methods are impractical. Our experimental results demonstrate that DFCA achieves performance comparable to centralized IFCA while operating under decentralized communication constraints, and it consistently outperforms decentralized FedAvg with momentum and FedSPD.

b) Discussion: Having demonstrated that DFCA effectively addresses decentralized clustered learning under heterogeneous data distributions, we now briefly discuss practical aspects that our design enables. The running-average update scheme can improve wall-clock efficiency in decentralized clustered FL. Unlike synchronous averaging, where all clients must wait for the slowest participant, running averages allow clients to incorporate neighbor updates at their own pace. Slow clients contribute less frequently but do not block progress for others, which is particularly advantageous in heterogeneous IoT or edge networks. This enables faster, more continuous progress and can reduce overall time-to-convergence. As our experiments did not include tests on random arrival times or communication delays, quantifying this effect is left for future work.

DFCA is well-suited for applications where data are inherently clustered and no central server is available or desirable, such as industrial IoT, sensor networks, or smart-city deployments. A deeper investigation of communication overheads and deployment characteristics is an important direction for future work.

REFERENCES

- [1] Mahmoud Assran, Nicolas Loizou, Nicolas Ballas, and Michael Rabbat. Stochastic gradient push for distributed deep learning, 2019.
- [2] Aurélien Bellet, Rachid Guerraoui, Mahsa Taziki, and Marc Tommasi. Personalized and private peer-to-peer machine learning, 2018.
- [3] Stephen Boyd, Arpita Ghosh, Balaji Prabhakar, and Devavrat Shah. Randomized gossip algorithms. *IEEE Transactions on Information Theory*, 52(6):2508–2530, 2006.
- [4] Sebastian Caldas, Sai Meher Karthik Duddu, Peter Wu, Tian Li, Jakub Konečný, H. Brendan McMahan, Virginia Smith, and Ameet Talwalkar. Leaf: A benchmark for federated settings, 2019.
- [5] Ken Chang, Niranjan Balachandar, Carson Lam, Darvin Yi, James Brown, Andrew Beers, Bruce Rosen, Daniel Rubin, and Jayashree Kalpathy-Cramer. Distributed deep learning networks among institutions for medical imaging. *Journal of the American Medical Informatics Association : JAMIA*, 25, 03 2018.
- [6] Vishnu Pandi Chellapandi, Antesh Upadhyay, Abolfazl Hashemi, and Stanislaw H. Żak. Decentralized federated learning: Model update tracking under imperfect information sharing. In *2024 IEEE International Conference on Big Data (BigData)*, pages 7697–7706, 2024.
- [7] Shuzhen Chen, Dongxiao Yu, Yifei Zou, Jiguo Yu, and Xiuzhen Cheng. Decentralized wireless federated learning with differential privacy, 2022.
- [8] Gregory Cohen, Saeed Afshar, Jonathan Tapon, and André van Schaik. Emnist: an extension of mnist to handwritten letters, 2017.
- [9] Elias Dritsas and Maria Trigka. Federated learning for iot: A survey of techniques, challenges, and applications. *Journal of Sensor and Actuator Networks*, 14(1), 2025.
- [10] Omar El-Rifai, Michael Ben Ali, Imen Megdiche, André Peninou, and Olivier Teste. A survey on cluster-based federated learning, 2025.
- [11] Avishek Ghosh, Jichan Chung, Dong Yin, and Kannan Ramchandran. An efficient framework for clustered federated learning, 2021.
- [12] Biyao Gong, Tianzhang Xing, Zhidan Liu, Wei Xi, and Xiaojiang Chen. Adaptive client clustering for efficient federated learning over non-iid and imbalanced data. *IEEE Transactions on Big Data*, 10(6):1051–1065, 2024.
- [13] István Hegedűs, Gábor Danner, and Márk Jelasity. Gossip learning as a decentralized alternative to federated learning. In José Pereira and Laura Ricci, editors, *Distributed Applications and Interoperable Systems*, pages 74–90, Cham, 2019. Springer International Publishing.
- [14] Chenghao Hu, Jingyan Jiang, and Zhi Wang. Decentralized federated learning: A segmented gossip approach, 2019.
- [15] Yixing Huang, Christoph Bert, Stefan Fischer, Manuel Schmidt, Arnd Dörfler, Andreas Maier, Rainer Fietkau, and Florian Putz. Continual learning for peer-to-peer federated learning: A study on automated brain metastasis identification, 2022.
- [16] Yeongwoo Kim, Ezeddin Al Hakim, Johan Haraldson, Henrik Eriksson, José Mairton B. da Silva Jr., and Carlo Fischione. Dynamic clustering in federated learning, 2020.
- [17] Anastasia Koloskova, Nicolas Loizou, Sadra Boreiri, Martin Jaggi, and Sebastian Stich. A unified theory of decentralized SGD with changing topology and local updates. In Hal Daumé III and Aarti Singh, editors, *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 5381–5393. PMLR, 13–18 Jul 2020.
- [18] Anastasia Koloskova, Sebastian U. Stich, and Martin Jaggi. Decentralized stochastic optimization and gossip algorithms with compressed communication. *CoRR*, abs/1902.00340, 2019.
- [19] Anastasia Koloskova, Sebastian U. Stich, and Martin Jaggi. Decentralized stochastic optimization and gossip algorithms with compressed communication. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019. Extended version in *JMLR*, 2020.
- [20] Alex Krizhevsky and Geoffrey Hinton. Learning multiple layers of features from tiny images. *Technical Report*, 2009.
- [21] Anusha Lalitha, Osman Cihan Kilinc, Tara Javidi, and Farinaz Koushanfar. Peer-to-peer federated learning on graphs. *arXiv preprint arXiv:1901.11173*, 2019.
- [22] Anusha Lalitha, Shubhanshu Shekhar, Tara Javidi, and Farinaz Koushanfar. Fully decentralized federated learning, 2018.
- [23] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 1998.
- [24] Chengxi Li, Gang Li, and Pramod K. Varshney. Federated learning with soft clustering. *IEEE Internet of Things Journal*, 9(10):7773–7782, May 2022.
- [25] Xiangru Lian, Ce Zhang, Huan Zhang, Cho-Jui Hsieh, Wei Zhang, and Ji Liu. Can decentralized algorithms outperform centralized algorithms? a case study for decentralized parallel stochastic gradient descent. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [26] Xiangru Lian, Wei Zhang, Ce Zhang, and Ji Liu. Asynchronous decentralized parallel stochastic gradient descent, 2018.
- [27] I-Cheng Lin, Osman Yagan, and Carlee Joe-Wong. Fed-SPD: A soft-clustering approach for personalized decentralized federated learning. In *The 41st Conference on Uncertainty in Artificial Intelligence*, 2025.
- [28] Guodong Long, Ming Xie, Tao Shen, Tianyi Zhou, Xianzhi Wang, and Jing Jiang. Multi-center federated learning: clients clustering for better personalization. *World Wide Web*, 26(1):481–500, June 2022.
- [29] Francesco Malandrino and Carla Fabiana Chiasserini. Federated learning at the network edge: When not all nodes are created equal, 2021.

- [30] Yishay Mansour, Mehryar Mohri, Jae Ro, and Ananda Theertha Suresh. Three approaches for personalization with applications to federated learning, 2020.
- [31] Othmane Marfoq, Chuan Xu, Giovanni Neglia, and Richard Vidal. Throughput-optimal topology design for cross-silo federated learning, 2020.
- [32] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-Efficient Learning of Deep Networks from Decentralized Data. In Aarti Singh and Jerry Zhu, editors, *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, volume 54 of *Proceedings of Machine Learning Research*, pages 1273–1282. PMLR, 20–22 Apr 2017.
- [33] Angelia Nedić and Alex Olshevsky. Stochastic gradient-push for strongly convex functions on time-varying directed graphs. *IEEE Transactions on Automatic Control*, 61(12):3936–3947, 2016. See also network optimization surveys (2018+).
- [34] Giovanni Neglia, Gianmarco Calbi, Don Towsley, and Gayane Vardoyan. The role of network topology for distributed machine learning. In *IEEE INFOCOM 2019 - IEEE Conference on Computer Communications*, pages 2350–2358, 2019.
- [35] Noa Onoszko, Gustav Karlsson, Olof Mogren, and Edwin Listo Zec. Decentralized federated learning of deep neural networks on non-iid data, 2021.
- [36] Christodoulos Pappas, Dimitris Chatzopoulos, Spyros Lalas, and Manolis Vavalis. Ipls : A framework for decentralized federated learning, 2021.
- [37] Abhijit Guha Roy, Shayan Siddiqui, Sebastian Pölsterl, Nassir Navab, and Christian Wachinger. Braintorrent: A peer-to-peer environment for decentralized federated learning, 2019.
- [38] Yichen Ruan and Carlee Joe-Wong. Fedsoft: Soft clustered federated learning with proximal local updating, 2022.
- [39] Felix Sattler, Klaus-Robert Müller, and Wojciech Samek. Clustered federated learning: Model-agnostic distributed multi-task optimization under privacy constraints, 2019.
- [40] Micah Sheller, Brandon Edwards, G. Reina, Jason Martin, Sarthak Pati, Aikaterini Kotrotsou, Mikhail Milchenko, Weilin Xu, Daniel Marcus, Rivka Colen, and Spyridon Bakas. Federated learning in medicine: facilitating multi-institutional collaborations without sharing patient data. *Scientific Reports*, 10, 07 2020.
- [41] Micah Sheller, G. Reina, Brandon Edwards, Jason Martin, and Spyridon Bakas. *Multi-institutional Deep Learning Modeling Without Sharing Patient Data: A Feasibility Study on Brain Tumor Segmentation: 4th International Workshop, BrainLes 2018, Held in Conjunction with MICCAI 2018, Granada, Spain, September 16, 2018, Revised Selected Papers, Part I*, volume 11383, pages 92–104. 01 2019.
- [42] Yandong Shi, Yong Zhou, and Yuanming Shi. Over-the-air decentralized federated learning, 2021.
- [43] Sebastian U. Stich. Local sgd converges fast and communicates little, 2019.
- [44] Tao Sun, Dongsheng Li, and Bao Wang. Decentralized federated averaging, 2021.
- [45] Yan Sun, Li Shen, and Dacheng Tao. Which mode is better for federated learning? centralized or decentralized, 2024.
- [46] Jianyu Wang, Anit Kumar Sahu, Gauri Joshi, and Soumya Kar. Matcha: A matching-based link scheduling strategy to speed up distributed optimization. *IEEE Transactions on Signal Processing*, 70:5208–5221, 2022.
- [47] Shuai Wang, Dan Li, Jinkun Geng, Yue Gu, and Yang Cheng. Impact of network topology on the performance of dml: Theoretical analysis and practical factors. In *IEEE INFOCOM 2019 - IEEE Conference on Computer Communications*, pages 1729–1737, 2019.
- [48] Liangqi Yuan, Yunsheng Ma, Lu Su, and Ziran Wang. Peer-to-peer federated continual learning for naturalistic driving action recognition, 2023.
- [49] Liangqi Yuan, Ziran Wang, Lichao Sun, Philip S. Yu, and Christopher G. Brinton. Decentralized federated learning: A survey and perspective, 2024.

APPENDIX

A. Convergence Analysis

We provide a proof template that reuses standard ingredients from clustered FL (e.g., [11]) for the assignment and from decentralized SGD/gossip (e.g., [25, 19, 3, 33]) for communication. Throughout, expectations are with respect to the stochasticity of data sampling and any communication randomness.

a) *Notation:* Clients are $M = \{1, \dots, N\}$, connected by an undirected graph $G = (M, E)$ with neighborhoods $\mathcal{N}_i$. The k cluster index set is $[k] = \{1, \dots, k\}$ and the (unknown) partition is $\{\mathcal{S}_1, \dots, \mathcal{S}_k\}$ with data distributions $\{\mathcal{D}_1, \dots, \mathcal{D}_k\}$. Client i stores parameters $(\theta_{i,1}, \dots, \theta_{i,k}) \in (\mathbb{R}^d)^k$. For cluster j , define the stacked vector $\Theta_j = (\theta_{1,j}, \dots, \theta_{N,j})$ and the network average $\bar{\theta}_j^t = \frac{1}{N} \sum_{i=1}^N \theta_{i,j}^t$. The client loss is

$$F_{\text{client}}(\theta_{i,j}, D_i) = \frac{1}{|D_i|} \sum_{d \in D_i} \mathcal{L}(\theta_{i,j}, d),$$

$$F_{\text{cluster}}(j) = \sum_{i \in \mathcal{S}_j} F_{\text{client}}(\theta_{i,j}, D_i),$$

$$F_{\text{global}} = \sum_{j=1}^k F_{\text{cluster}}(j).$$

We measure per-cluster *disagreement* by

$$\text{Disp}_j^t = \frac{1}{N} \sum_{i=1}^N \|\theta_{i,j}^t - \bar{\theta}_j^t\|^2. \quad (14)$$

b) *Assumption 3 Extension:* Let X_t be the row-stacked matrix of client models (for any fixed cluster index j), let $\bar{x}_t = \frac{1}{N} \mathbf{1}^\top X_t$ be the network average, and define the disagreement

$$E_t = X_t - \mathbf{1} \bar{x}_t^\top. \quad (15)$$

Synchronous mixing. There exists a symmetric, doubly-stochastic matrix W respecting G with spectral gap $1 - \lambda > 0$ such that

$$\|E_{t+1}\| * F = \|X_t W - \mathbf{1}\bar{x}_t^\top\| * F \leq \lambda \|E_t\|_F. \quad (16)$$

Asynchronous (gossip) mixing. Let $\{W_t^{(j)}\}$ be the sequence of row-stochastic mixing matrices arising from pairwise/asynchronous gossip.

Assume edges are activated with bounded delays and there exists a window size B and a contraction factor $\tilde{\lambda} \in (0, 1)$ such that

$$\mathbb{E}[\|E_{t+B}\| * F^2 \mid E_t] \leq \tilde{\lambda} \|E_t\| * F^2. \quad (17)$$

These are standard consensus contraction assumptions in decentralized optimization (i.e. [25, 17]).

Lemma 1 (Assignment is descent for F_{global}). *Conditioned on parameters $\{\theta_{i,j}^t\}$, the assignment step does not increase F_{global} :*

$$\sum_{i=1}^N \min_j F_{\text{client}}(\theta_{i,j}^t, D_i) \leq \sum_{i=1}^N F_{\text{client}}(\theta_{i,c_{t-1}(i)}^t, D_i). \quad (18)$$

Proof. Pointwise argmin over j per client i can only reduce the sum; cf. [11]. $\square$

Lemma 2 (Local SGD descent with disagreement penalty). *Let $\gamma \leq 1/L$. Then, conditioned on Θ^t ,*

$$\mathbb{E}\left[F_{\text{cluster}}(j; \bar{\theta}_j^{t+\frac{1}{2}}) \mid \Theta^t\right] \leq F_{\text{cluster}}(j; \bar{\theta}_j^t) - \frac{\gamma}{2} \|\nabla F_{\text{cluster}}(j; \bar{\theta}_j^t)\|^2 + \gamma^2 L \left(\sigma^2 + L^2 \text{Disp}_j^t\right). \quad (19)$$

Proof. Apply the smoothness descent lemma to the cluster-sum objective using unbiased gradients, and decompose the error into stochastic noise σ^2 and a consensus term proportional to Disp_j^t . This form follows standard decentralized SGD analyses, e.g. [25, 43, 17]. $\square$

Lemma 3 (Gossip preserves averages and contracts disagreement). *For each j , $\bar{\theta}_j^{t+1} = \bar{\theta}_j^{t+\frac{1}{2}}$. Moreover, in the synchronous (fixed W) case,*

$$\mathbb{E}[\text{Disp}_j^{t+1} \mid \Theta^{t+\frac{1}{2}}] \leq \lambda^2 \text{Disp}_j^{t+\frac{1}{2}}. \quad (20)$$

In the asynchronous case, for some window B and $\tilde{\lambda} \in (0, 1)$, $\mathbb{E}[\text{Disp}_j^{t+B}] \leq \tilde{\lambda}^2 \text{Disp}_j^t$.

Proof. Average preservation follows from row-stochasticity (and doubly-stochasticity in the synchronous case). Disagreement evolution is governed by multiplication with $W_t^{(j)}$; contraction follows from the spectral gap (synchronous) or joint-connectivity arguments for randomized gossip [3, 33]. $\square$

Lemma 4 (Assignment stabilization). *Under (A1)–(A5) with sufficiently small γ , there exists a finite τ such that $c_t(i) = c_*(i)$ for all i and all $t \geq \tau$.*

Proof sketch. By Lemmas 2–3, the averages $\{\bar{\theta}_j^t\}$ descend and the disagreements Disp_j^t contract, so all client copies tracking

a fixed j enter and remain in a neighborhood of θ_j^* . Within this neighborhood, separability (A5) enforces a unique, correct argmin, hence stable assignments; cf. [11]. $\square$

Theorem 2 (Convergence of DFCA from Theorem 1). *Assume (A1)–(A5), choose $\gamma \leq c/L$ for a small numerical constant c , and let λ (resp. $\tilde{\lambda}$) be the consensus factor in the synchronous (resp. async) case. Then:*

- (i) (Pre-stabilization) F_{global}^t is non-increasing in expectation across assignment and local steps. The disagreements $\{\text{Disp}_j^t\}$ remain bounded and contract at rate λ (or $\tilde{\lambda}$ over windows).
- (ii) (Stabilization) There exists $\tau < \infty$ such that $c_t(i) = c_*(i)$ for all $t \geq \tau$.
- (iii) (Post-stabilization) For $t \geq \tau$, DFCA is k independent copies of decentralized SGD on $F_{\text{cluster}}(j)$.
 - Under (PL) for some $\mu > 0$,

$$\mathbb{E}\left[F_{\text{global}}^{\tau+T} - F_{\text{global}}^*\right] \leq (1 - \mu\gamma/2)^T C_0 + O\left(\frac{\gamma\sigma^2}{\mu}\right) + O\left(\frac{\gamma L}{1-\lambda} \sigma^2\right), \quad (21)$$

with C_0 depending on the gap at $t = \tau$; in async, replace $(1 - \lambda)$ by the windowed $(1 - \tilde{\lambda})$.

- Under (Cvx),

$$\frac{1}{T} \sum_{t=\tau}^{\tau+T-1} \sum_{j=1}^k \mathbb{E} \|\nabla F_{\text{cluster}}(j; \bar{\theta}_j^t)\|^2 \leq O\left(\frac{F_{\text{global}}^\tau - F_{\text{global}}^*}{\gamma T}\right) + O(\gamma L \sigma^2) + O\left(\frac{\gamma L}{1-\lambda} \sigma^2\right), \quad (22)$$

and choosing $\gamma = \Theta(1/\sqrt{T})$ yields the usual $O(1/\sqrt{T})$ rates (with the consensus penalty).

Proof sketch. Combine Lemma 1 (assignment descent), Lemma 2 (SGD descent with a disagreement term), and Lemma 3 (average preservation and disagreement contraction). Lemma 4 yields finite-time stabilization, after which each cluster index j follows a standard decentralized SGD recursion; apply known rates under PL or convexity and sum over j . $\square$

c) Remarks: (i) *Batch vs. sequential aggregation.* The sequential “running average” update $\theta \leftarrow \frac{r}{r+1}\theta + \frac{1}{r+1}\theta_{\text{new}}$ implements a valid stochastic gossip step; the windowed contraction in Lemma 3 covers it. (ii) *Initialization.* Global initialization (DFCA-GI) sets $\text{Disp}_j^0 = 0$ and typically reduces τ ; local initialization (DFCA-LI) only changes constants. (iii) *Clients not training j .* They still mix $\theta_{i,j}$ by applying $W_t^{(j)}$ to their current value; average preservation and contraction remain valid.



Jonas Kirch Jonas Kirch completed his BS in Computer Science at TU Dortmund University in 2025 and currently pursues his MS in Computer and Mathematical Sciences at Tohoku University. He was formally affiliated with the Fraunhofer Institute for Software and Systems Engineering as a research assistant for the department of mobility and smart cities. His research interests include Machine Learning, Federated Learning, Decentralized Optimization, Wireless Communication and Internet of Things.



Sebastian Becker Sebastian Becker is a Research Associate at the Fraunhofer Institute for Software and Systems Engineering (ISST) in Dortmund, Germany. He studied mathematics and computer science at the Heinrich-Heine University in Düsseldorf where he received his BSc in computer science in 2019, his MSc in computer science in 2022 and his BSc in mathematics in 2023. All three theses involved different aspects of machine learning and deep learning. Since then, Sebastian Becker began studies for his PhD which focuses mainly on decentralized federated learning. Besides that, his interests also include all aspects of machine learning and federated learning as well as data spaces and other data sharing technologies.



Tiago Koketsu Rodrigues Tiago Koketsu Rodrigues [M'15, SM'25] (koketsu.rodrigues.tiago.c4@tohoku.ac.jp), previously Tiago Gama Rodrigues, has been an associate professor at Tohoku University since June 2025. He received his Bachelor's Degree in Computer Science from the Federal University of Piauí, in Brazil, in 2014, his M.Sc. degree from Tohoku University, in Japan, in 2017 and his Ph.D. from the same institution in 2020. From 2020 to 2025, he was an assistant professor at

Tohoku University. He was the recipient of the 2018 Best Paper Award from IEEE Transactions on Computers, the 2020 Tohoku University President Award, and the IEEE Communications Society Asia-Pacific Region 2022 Outstanding Young Researcher Award, among others. From 2017 to 2020, he was the System Administrator of the IEEE Transactions on Vehicular Technology. He is an editor at IEEE Network, IEEE Transactions on Vehicular Technology, and IEEE Transactions on Emerging Topics in Computing, and an Associate Editor-in-Chief at IEEE Internet of Things Journal.



Stefan Harmeling Stefan Harmeling is Professor for Artificial Intelligence at the TU Dortmund, Germany. Dr Harmeling studied mathematics and mathematical logic at the University of Münster (Dipl Math 1998) and computer science with an emphasis on artificial intelligence at Stanford University (MSc 2000). During his doctoral studies he was at the Fraunhofer Institute FIRST (Dr rer nat 2004). Thereafter he was for two years a Marie Curie Fellow at the University of Edinburgh (2005-2007). In 2007 he joined the Max Planck Institute of Biological Cybernetic, which was later included in the new Max Planck Institute of Intelligent Systems in Tübingen. In 2014 he became Professor for Machine Learning at HHU Düsseldorf and since March 2022 he is Professor for Artificial Intelligence at TU Dortmund, Germany. His interests include all aspects of machine learning and artificial intelligence, currently in particular, deep learning, reinforcement learning, and application in science.